DEVELOPER TASK ASSIGNMENT

Google Vids
Video Generation Automation

Python + Selenium + HTML/CSS | Local Downloads | Multi-Part Workflow | Error Recovery

Task Type	Browser automation / desktop-friendly local tool
Primary Goal	Generate videos in Google Vids from one or more script parts and their reference images, then save completed outputs locally with predictable filenames.
Priority	Reliability, recoverability, clear logs, and no loss of already completed work.
Important Constraint	Google Vids UI, account entitlements, quotas, and automation permissions may change. The implementation must verify what is actually available rather than assume every step can be automated.

Deliverable expectation: a maintainable automation project, setup instructions, test evidence, and a handover document. Do not treat this as a blind click-script; it must detect failures and preserve progress.

1. Project Objective

Build a Python-based automation tool that accepts video scripts and matching reference images, opens Google Vids through a supported browser workflow, generates a video for a single script or manages multiple parts in one project where the product supports it, and downloads completed output to a local folder.

- **Single-part job:** submit one script with its reference image(s), generate the video, wait for completion, download it, and save it using the requested filename.
- **Multi-part job:** submit each script part with its own reference image(s), add/organize parts in the same Vids project if supported by the current UI, generate all intended parts, and download the final output or individual outputs according to the selected mode.
- **Quota/limit reached:** stop further generation safely, save/download all completed outputs that are available, persist job state, and show exactly which parts are complete, pending, or blocked.
- **Recovery:** after a browser crash, timeout, or restart, resume from the last confirmed state without silently regenerating or overwriting finished files.

2. Required Technology

Technology	Purpose
Python 3.11+ (recommended)	Workflow orchestration, validation, retries, state management, logs, and downloads.
Selenium 4	Browser UI automation only where permitted and stable; explicit waits and locator strategy required.
HTML/CSS	Simple local dashboard/form for job creation, progress, file selection, and status display.
SQLite + JSON	SQLite for durable job/part state; JSON for importable job manifests and configuration.
Logging + tests	Rotating logs, structured error records, unit tests, and end-to-end smoke tests.

Architecture note: HTML/CSS alone do not provide backend automation. Use a small local Python web app (for example Flask) or a Python desktop launcher to serve the UI and invoke the workflow. The developer should keep browser automation and UI code separated.

3. User Workflow

Create a job

User enters a project name, output folder, naming convention, and mode: single video, multi-part project, or separate clips.

Add parts

For each part, user supplies part number/title, script text, and reference image(s). Support adding, removing, reordering, and validating parts.

Validate inputs

Check scripts are non-empty, images exist and can be opened, filenames are safe, output directory is writable, and required fields are filled.

Review plan

Show a summary before starting: part count, script names, image files, intended output names, and selected generation mode.

Run automation

Open Google Vids, confirm login/project state, submit content using current supported UI steps, and track each transition.

Monitor generation

Wait for visible completion indicators; do not use fixed long sleeps as the only signal. Display current part, elapsed time, and status.

Download and verify

Download completed videos, wait for the browser download to finish, verify file exists and has non-zero size, then rename safely.

Finish or resume

Write a final report. If interrupted, persist state and offer Resume / Retry failed part / Open output folder.

4. Input Format and File Naming

Recommended local folder structure:

```
project_root/
  app.py
  requirements.txt
  config.example.json
  automation/
    vids_controller.py
    workflow.py
    downloads.py
    validators.py
    state_store.py
  ui/
    templates/index.html
    static/style.css
  data/jobs.sqlite3
  input/
    Project_A/part_01/script.txt
    Project_A/part_01/reference.png
    Project_A/part_02/script.txt
    Project_A/part_02/reference.png
  output/Project_A/
  logs/automation.log
  tests/
```

Suggested naming: ProjectName_Part01.mp4 for individual clips; ProjectName_Final.mp4 for a combined/exported project. Preserve the original downloaded filename in the job record. If a target file already exists, never overwrite silently—use a version suffix or ask the user.

Optional job manifest example:

```
{
  "project_name": "Project_A",
  "mode": "multi_part",
  "output_dir": "output/Project_A",
  "parts": [
    {"part": 1, "script_file": "input/Project_A/part_01/script.txt", "reference_images":
["input/Project_A/part_01/reference.png"]},
    {"part": 2, "script_file": "input/Project_A/part_02/script.txt", "reference_images":
["input/Project_A/part_02/reference.png"]}
  ]
}
```

5. Single-Part vs Multi-Part Requirements

A. Single script / one video

- Create one job and one part record.
- Attach or select the corresponding reference image(s) as supported by the current Google Vids interface.
- Start generation and detect success, failure, timeout, sign-in prompts, or quota/limit messages.
- Download only after the UI confirms the output is ready; verify and rename the local file.
- Mark the job complete only after the local file is verified.

B. Multiple scripts / part-wise generation

- Display a reorderable list of parts and retain the association between each script and its reference image(s).
- Use the same Google Vids project and add parts/scenes only if the current product supports the intended workflow; otherwise clearly report the limitation and use a safe per-part workflow.
- Maintain separate states for each part: Pending, Submitted, Generating, Ready, Downloaded, Failed, or Blocked by quota.
- Do not assume that a UI element named 'part' or a multi-scene editor is available; confirm it during a discovery/prototype phase.
- When a combined final export is supported, export it and verify it. If only individual outputs are available, save each clip separately and report that no combined export was produced.

C. Reference image handling

- Allow common formats such as PNG, JPG/JPEG, and WEBP; validate actual file readability.
- Show thumbnails and filenames before submission.
- Do not silently substitute an image, attach the wrong image to a part, or change the user's script.
- If image upload fails, retry within a limit and keep the part pending rather than continuing with incomplete inputs.

6. Error Handling and Recovery (Mandatory)

Failure scenario	Required behavior
Free quota / usage limit reached	Detect visible limit message; stop submitting new generations; download and verify all completed outputs; persist pending work and show remaining parts as blocked. Never claim a quota can be bypassed.
Network timeout / slow page	Use explicit waits, bounded retries with backoff, and a maximum timeout; take a screenshot and log the current step on final failure.
Login / session expired / CAPTCHA	Pause and request user action. Do not store passwords in source code, automate CAPTCHA solving, or bypass access controls.
Element not found / UI changed	Fail safely, capture screenshot and page context, log locator/step, and avoid repeated blind clicks. Use resilient selectors and a small page-object layer.
Generation failed	Mark only that part failed; keep other completed files; allow retry of the failed part without resetting the whole job.
Download interrupted	Detect incomplete/temp files; wait or retry download; verify final extension and non-zero size before marking downloaded.
Duplicate filename	Never overwrite silently. Create a deterministic suffix such as _v2 or request user confirmation.
Browser / Python crash	Persist state after every confirmed milestone; on restart, reconcile database state with local files and resume safely.
Disk full / permission denied	Stop new generation, show the path and error, preserve logs/state, and retry only after user fixes storage/permissions.

7. State Tracking and Safe Resume

Persist state in SQLite (or an equivalent durable store). Minimum records:

- Job: job_id, project_name, mode, created_at, updated_at, overall_status, output_dir, last_error.
- Part: part_id, job_id, sequence, script_path or script text reference, image paths, status, retry_count, Vids project/part reference if available, output_path, error_message.
- Event log: timestamp, job_id, part_id, step, severity, message, screenshot_path where applicable.

Commit state after each meaningful transition: validated, submitted, generation started, generation ready, download started, download verified. Resume logic must check the actual local file and stored state before deciding whether to regenerate.

8. UI Requirements

- Clean HTML/CSS page with project name, mode selector, output folder, and add-part controls.
- Each part card: script text/file input, reference image upload, thumbnail, order controls, validation status, and remove action.
- Buttons: Validate Job, Start, Pause/Stop Safely, Resume, Retry Failed Parts, Open Output Folder (if platform allows), Export Report.
- Progress panel: overall status, part-by-part status, current step, retry count, last error, and completed/total count.
- Clear warnings for Google sign-in, permission prompts, usage limits, and any unsupported workflow.

- Keep the UI responsive; long-running Selenium tasks should run in a background worker/thread or task queue.

9. Selenium Implementation Standards

- Use explicit waits (WebDriverWait) and expected conditions; avoid relying on time.sleep() as the primary synchronization mechanism.
- Use stable accessible attributes/labels where possible; centralize selectors and page actions in a page-object/controller module.
- Set a dedicated download directory and configure browser preferences so downloads can be tracked predictably.
- Keep ChromeDriver/browser compatibility managed; document the supported browser and version.
- Do not hardcode selectors throughout the workflow. Add diagnostics for missing/changed elements.
- Use screenshots and concise DOM/page diagnostics on failure, but do not log sensitive account information.
- Respect Google product terms, account restrictions, usage limits, and organizational policy. If the UI or policy does not permit a step, stop and report it rather than evading the restriction.

10. Google Vids Feasibility Check (Must be First Milestone)

Before implementing the full workflow, the developer must run a short proof of concept using the intended Google account and current Google Vids UI.

- Confirm the account can access Google Vids and identify the exact available generation features and limits.
- Confirm whether script + reference-image input is supported in the required way.
- Confirm whether multiple parts/scenes can be created in one project and whether a single combined video can be exported.
- Confirm whether output download is available and what format is produced.
- Confirm whether browser automation is allowed and stable for the required steps. If a step requires manual confirmation, mark it as a human-in-the-loop step.
- Deliver a short feasibility report with screenshots of the normal workflow, quota/limit behavior if naturally encountered, and a clear list of supported vs unsupported steps.

Important: The automation must not assume that Google Vids exposes a public API or guarantees a particular export/download control. Google may change its UI, features, and account-specific limits. The task is to build against verified current behavior, not to bypass restrictions.

11. Security, Privacy, and Data Handling

- Do not store Google passwords, recovery codes, session cookies, or authentication tokens in plain text or source control.
- Use the user's existing browser profile or an approved sign-in flow only if permitted and secure; do not copy private session data into logs.
- Store scripts and reference images locally; make it clear where files are saved and how to remove job data.
- Sanitize project names and filenames to prevent path traversal and invalid file paths.
- Keep logs free of script content if scripts may contain confidential information; log IDs and step names instead.
- Do not bypass CAPTCHA, quota limits, account controls, or other technical restrictions.

12. Testing and Acceptance Criteria

- A single-script job can be created, validated, generated (if supported), downloaded, renamed, and marked complete.
- A multi-part job preserves the correct script/image mapping and part order.
- Every downloaded file is checked for existence and non-zero size before success is recorded.
- A duplicate filename never causes silent overwrite.
- When quota/limit is detected, no new generation is attempted; all completed available outputs are saved and pending parts remain resumable.
- A failed part can be retried without regenerating already downloaded parts.
- Restarting the app recovers saved job/part state and reconciles completed files.
- Timeout, network loss, missing element, login prompt, download failure, and disk error are logged with actionable messages.
- The UI clearly shows completed, pending, failed, and blocked parts.
- Developer provides automated tests for validators/state management and an end-to-end smoke test for the supported browser workflow.
- Setup instructions work on a clean Windows machine, including dependencies, browser setup, configuration, and launch command.
- No secrets are committed; logs do not expose credentials or session tokens.

13. Suggested Milestones

Milestone	Work	Evidence
M1 — Feasibility	Verify Google Vids workflow, supported inputs, multi-part behavior, export/download, and policy constraints.	Short report + screenshots + go/no-go list.
M2 — Input UI + validation	Build HTML/CSS job form and script/image validation.	Demo with one part and 3+ parts.
M3 — Automation core	Selenium controller, waits, download handling, and error diagnostics.	Recorded demo + logs.
M4 — State + resume	SQLite state, retry policy, quota-stop handling, resume workflow.	Simulated failure/restart test report.
M5 — QA + handover	Acceptance tests, documentation, packaging, clean-machine setup.	Source, tests, README, final demo.

14. Final Handover Checklist

- Complete source code with clean module separation.
- requirements.txt and configuration example with no secrets.
- README with installation, setup, run, troubleshooting, and limitations.
- Sample input project with at least three parts and sample reference images.
- Automated test results and a test report for quota/limit simulation.
- Demo video showing one-part workflow, multi-part workflow, a simulated limit/error, saving completed work, and resuming.
- List of known limitations and which steps require user interaction.
- Confirm all output files and logs are saved to the documented local folders.

15. Task Completion Definition

The task is complete only when the developer demonstrates the supported workflow end-to-end, shows that completed downloads survive a quota/limit or interruption, proves resume/retry behavior, supplies source code and setup documentation, and documents any Google Vids feature or policy limitation found during feasibility testing. If the current product does not support a requested step, the developer must report it and propose a compliant alternative instead of pretending it works.

Suggested assignment note: Start with Milestone 1 and do not spend time building the full Selenium flow until Google Vids inputs, multi-part editing, and downloads have been verified on the target account.